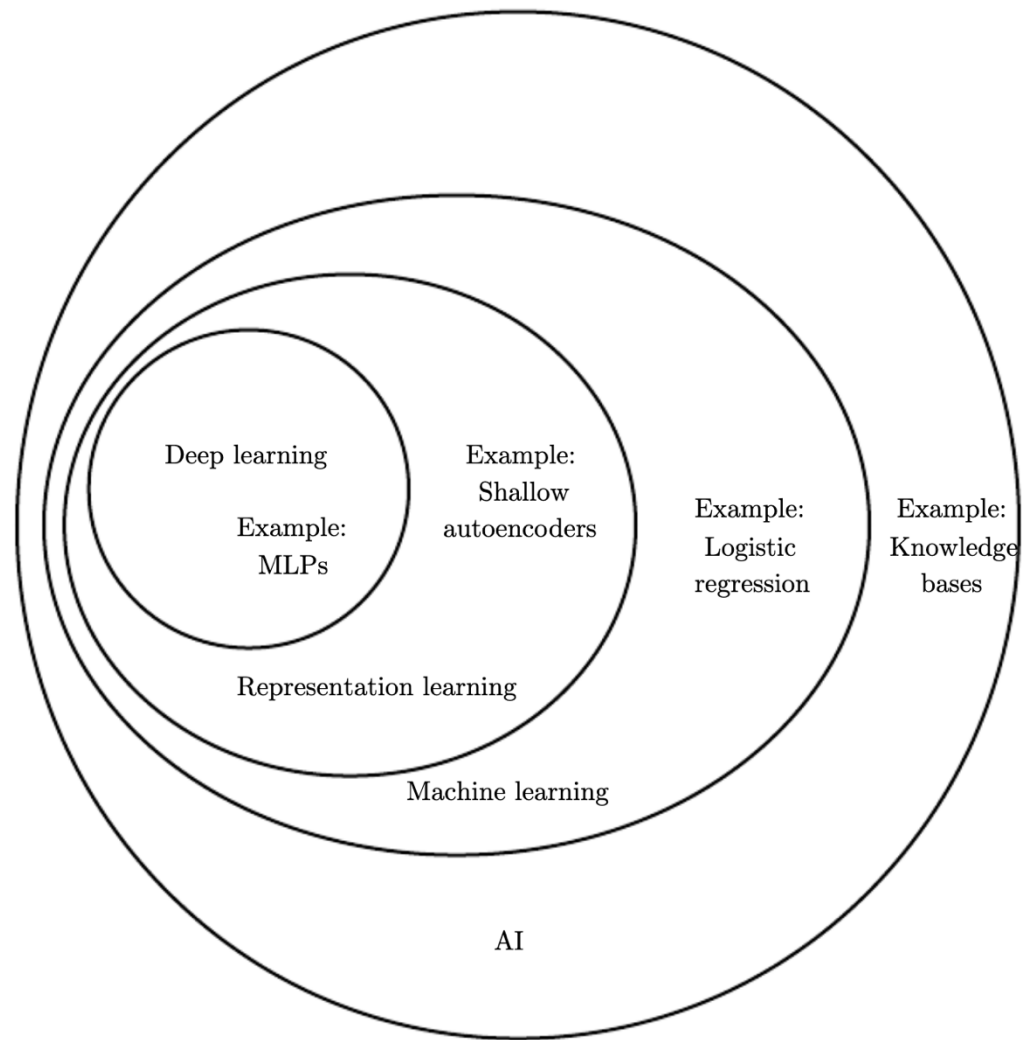


Introduction to Deep Learning

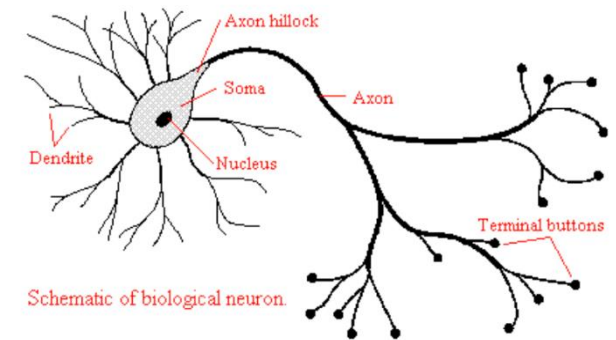
Introduction to Deep Learning
Deep Learning and Malware analysis

AI, Machine Learning, and Deep Learning: The Relationship



What is Deep Learning?

- Deep learning is a subset of machine learning driven by multilayered neural networks whose design is inspired by the structure of the human brain's neural circuits. whose functioning is driven by the complex transmission of chemical and electrical signals across distributed networks of nerve cells (neurons). In deep learning, the analogous “signals” are the weighted outputs of many nested mathematical operations, each performed by an artificial “neuron” (or node), that collectively comprise the neural network.
- The artificial neural networks of deep learning models comprise many interconnected layers of “neurons” that each perform a mathematical operation.
- Deep learning models are most commonly trained through supervised learning on labeled data to perform regression and classification tasks



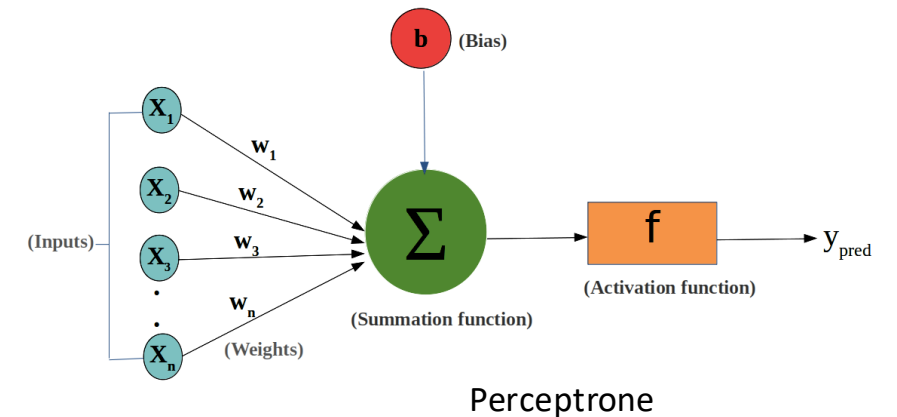
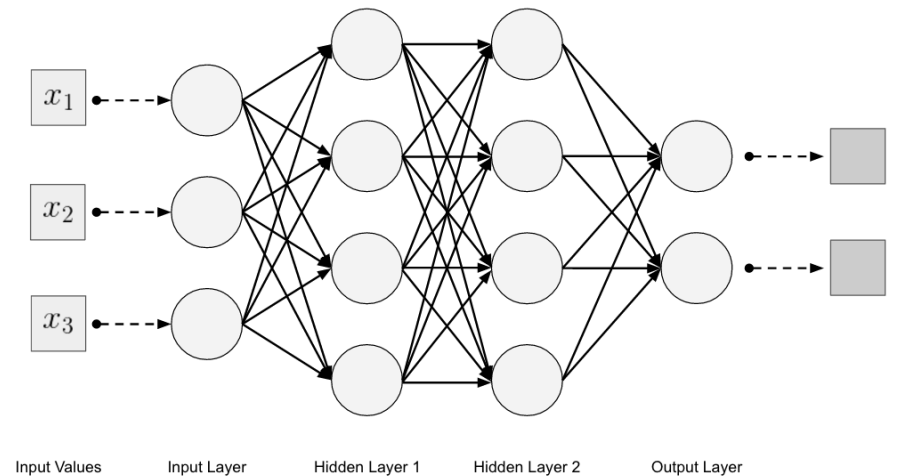
Why is it called "Deep"?

Because the model performs **multiple successive transformations**:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \hat{y}$$

where:

- x = input data
- h_1 = first learned representation
- h_2 = deeper representation
- h_3 = even higher-level representation
- $\hat{y}$ = final prediction



Perceptrone ..

Core components:

1. Inputs $(x_1, x_2, \dots, x_n)$

These are the features or measurable attributes of a data point that the perceptron uses to make a decision. Each input provides a signal that contributes to the final output.

Example: For an OR gate, the inputs are binary: $(x_1, x_2) \in \{0, 1\}^2$

Inputs themselves have no inherent influence unless multiplied by weights.

2. Weights $(w_1, w_2, \dots, w_n)$

- Weights determine how strongly each input contributes to the prediction. A larger weight means the corresponding input has a higher impact.
- Weights are learned during training, adjusting based on errors.
- They act like importance scores for each feature.

3. Bias (b)

- The bias is a constant value added to the weighted sum to shift the decision boundary.
- It allows the perceptron to classify correctly even when all input features are zero.
- Bias ensures the model is not forced to pass the decision boundary through the origin.

Difference Between Weights and Bias

- Weights control how much each input influences the output.
- Bias controls when the perceptron activates, independent of any input.
- Mathematically, Weights tilt the line and Bias shifts the line up/down or left/right.

Perceptrone ..

4. Net Input (Weighted Sum)

This is the combined effect of all inputs and their weights:

$$z = \sum_{i=1}^n w_i x_i + b$$

Represents the activation strength before passing through the activation function.

If z is high or low enough, it determines the final class.

5. Activation Function (Step Function) (others: ReLU, Sigmoid, Tanh, etc.)

The activation function converts the numerical input into a binary output:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

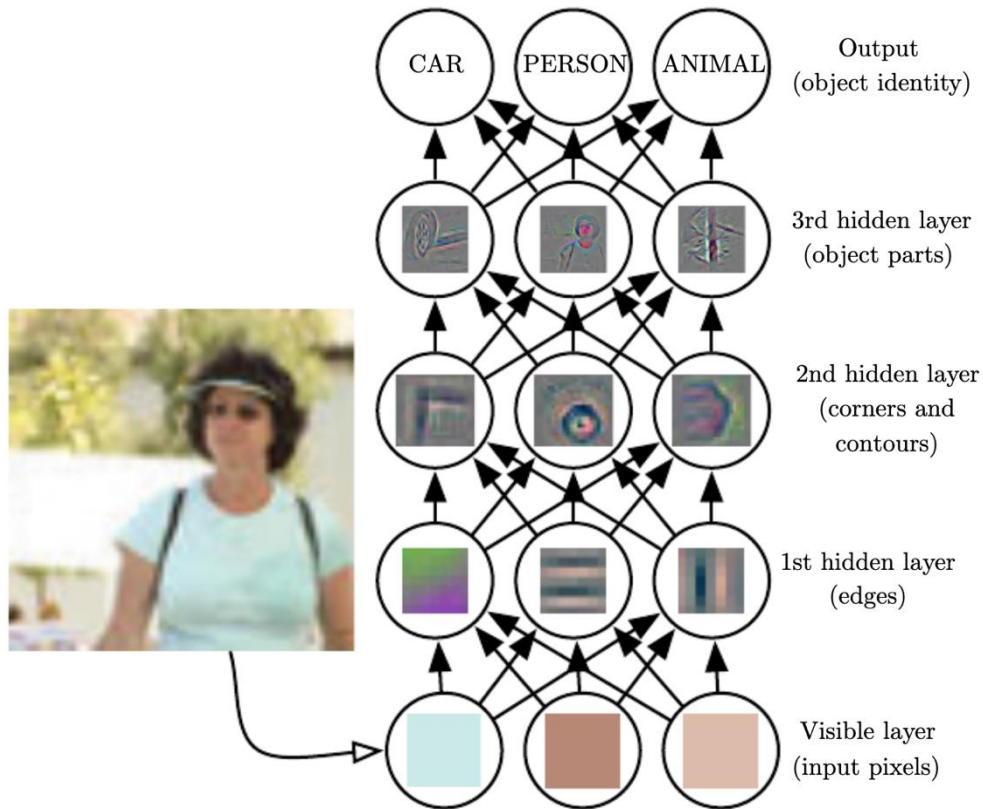
- It introduces non-linearity in the decision-making, although the decision boundary remains linear.
- Output is always 0 or 1 making perceptrons suitable for binary classification.

Perceptrone ..

Common activation functions

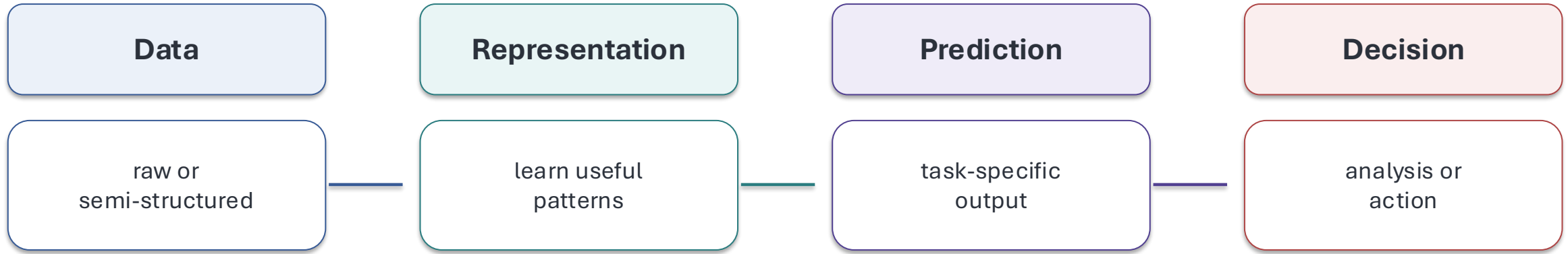
Function	Definition	Main idea
Step	$f(z) = 1 \text{ if } z \geq 0, \text{ otherwise } 0$	Binary on/off decision
ReLU	$f(z) = \max(0, z)$	Keeps positive values, removes negative values
Sigmoid	$f(z) = \frac{1}{1 + e^{-z}}$	Maps values to 0–1
Tanh	$f(z) = \tanh(z)$	Maps values to –1–1

How Deep Learning works ?



- **Input layer:** The model receives raw data, such as the pixels of an image.
- **Hidden layers:** Each layer learns increasingly **abstract representations** of the input, starting with simple features such as edges, then corners and contours, and finally object parts.
- **Hierarchical learning:** Each layer builds on the representation learned by the previous layer, breaking a complex recognition problem into a sequence of simpler transformations.
- **Output layer:** The learned representations are combined to identify the final object, such as car, person, or animal.

Deep Learning : Learning Representations from Data



- “Deep” refers to composing multiple transformation stages.
- The system can learn internal representations rather than depending entirely on manually engineered features.

Traditional ML vs. Deep Learning

The distinction is mainly about representation learning, not simply layer count.

Traditional machine learning

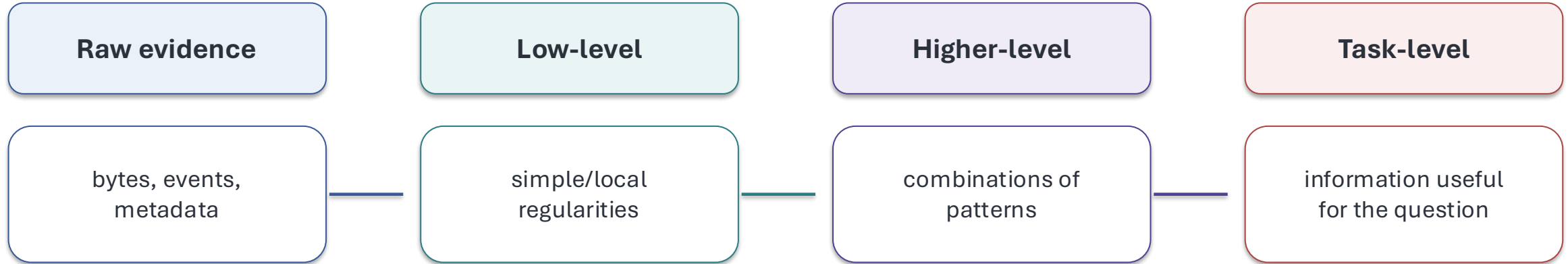
- Raw data
- Human-designed features
- Feature selection / transformation
- Learning algorithm
- Prediction

Deep learning

- Raw / minimally processed data
- Learned representation
- Multiple levels of abstraction
- Task-specific optimization
- Prediction

Deep learning does not remove domain knowledge; it shifts part of the representation-building process into learning.

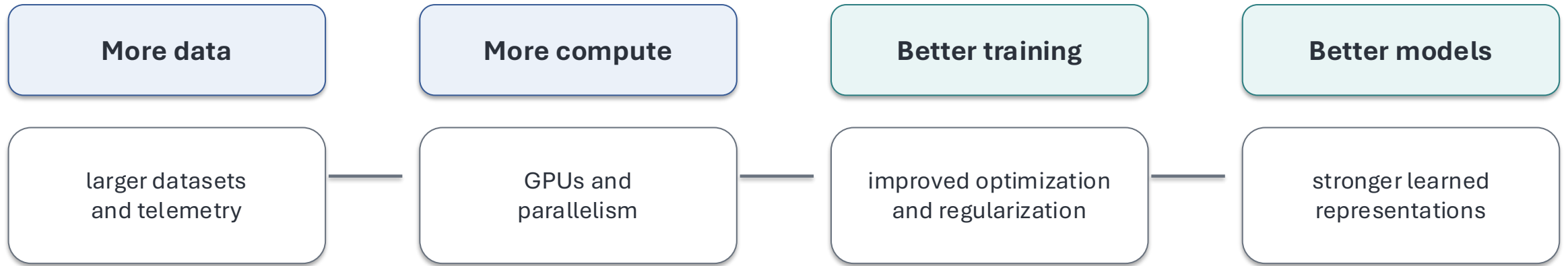
Why representation learning matters?



A useful representation preserves information relevant to the task while reducing irrelevant variation.

- In malware analysis, raw observations contain both signal and noise.
- Learning aims to emphasize patterns associated with maliciousness, behavior, or relationships among samples.

Why deep learning became practical



These factors reinforce one another: data enables learning, compute enables scale, and better training makes the scale useful.

- Security environments increasingly generate large collections of files, events, and telemetry.
- This creates opportunities for data-driven malware analysis.

Major Types Deep Learning by neural-network architecture:

- **Feedforward Neural Networks (FNN/MLP)**
Information moves from input → hidden layers → output.
- **Convolutional Neural Networks (CNNs)**
Designed to learn local/spatial patterns. They can be applied to images, byte representations, and other structured inputs.
- **Recurrent Neural Networks (RNNs)**
Designed for sequential data where order matters, such as API-call sequences.
- **Long Short-Term Memory (LSTM)**
A type of RNN designed to capture longer-term dependencies in sequences.
- **Autoencoders**
Learn compact representations by encoding data and reconstructing it.
- **Transformers**
Use attention mechanisms to model relationships between elements in a sequence or set of inputs.

Note: there are many others, but these what we will cover in this course

What is malware?

MALWARE

Software or code intended to perform unauthorized, harmful, deceptive, or otherwise unwanted actions.

Virus

replication

Worm

propagation

Trojan

disguised delivery

Ransomware

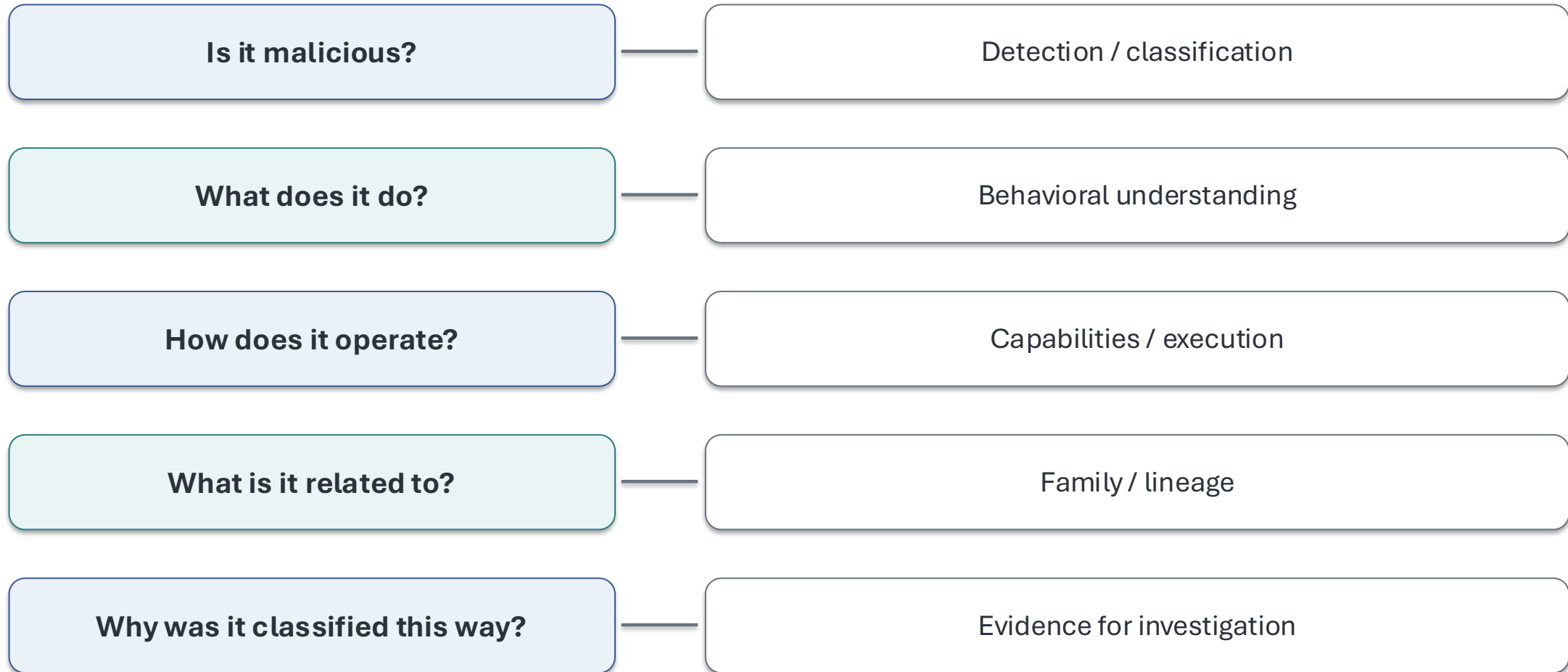
access denial

Spyware

information collection

A sample may exhibit multiple behaviors; categories are useful descriptors rather than perfect boundaries.

What does Malware Analysis try to answer?



Static and Dynamic Malware Analysis

Static analysis

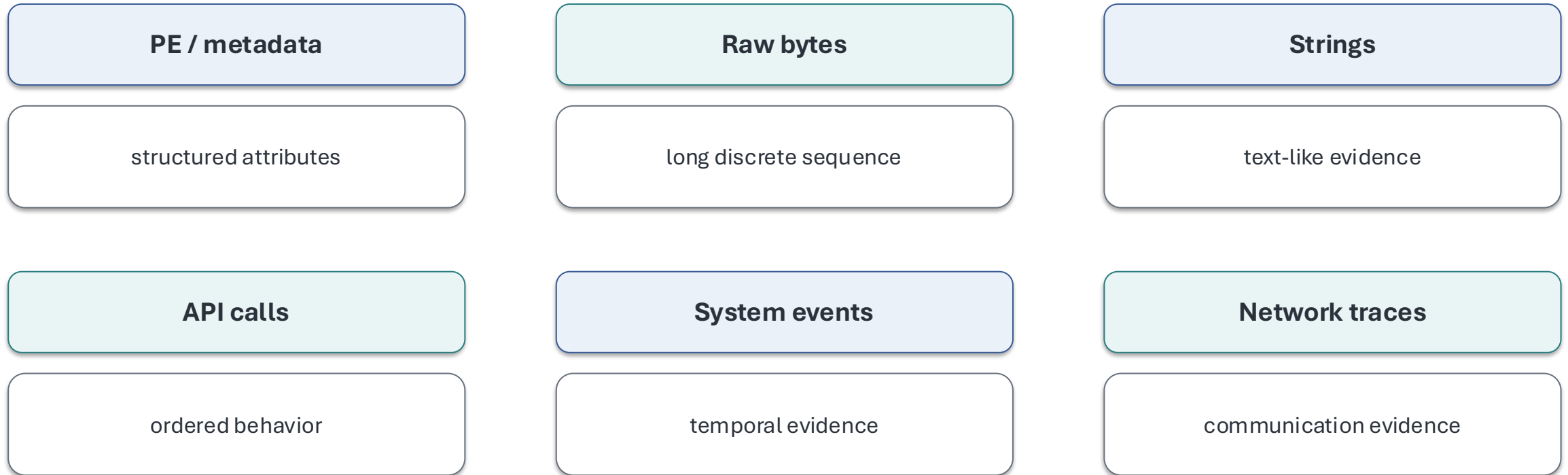
- Sample is examined without executing it.
- Possible evidence: headers, metadata, strings, imported functions, byte-level information.
- Often easier to scale.
- May miss behavior that only appears during execution.

Dynamic analysis

- Sample executes in a controlled environment.
- Possible evidence: API calls, processes, files, network activity, memory behavior.
- Captures behavior directly.
- Can be costly, evasive, and difficult to reproduce.

Deep learning can operate on either type of evidence; the representation must match the question.

Malware is heterogeneous data



There is no single “malware input.” The representation depends on available evidence and the analysis question.

Why malware is a difficult learning problem

High variability

Samples from the same category can look very different.

Obfuscation

Useful evidence may be hidden or transformed.

Adversarial adaptation

Attackers can intentionally change observable characteristics.

Label uncertainty

Ground truth can be incomplete, noisy, or time-dependent.

Class imbalance

Some families or behaviors are rare.

Distribution shift

Future malware may differ from historical data.

Why use Deep Learning for Malware Analysis?

Representation learning

Reduce dependence on fixed manual feature sets.

Multiple evidence types

Potentially learn from different modalities.

Scale

Learn from large collections of files and telemetry.

Reuse

Representations may support related tasks.

Nonlinear patterns

Capture complex relationships in observations.

Automation

Support large-scale triage and analyst workflows.

Deep Learning is not automatically better

Potential strengths

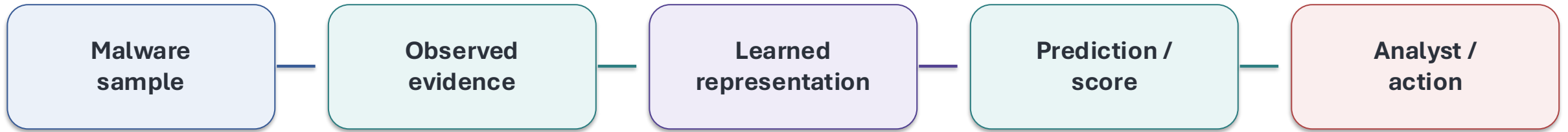
- Learns representations
- Handles high-dimensional evidence
- Models nonlinear relationships
- Can scale with data
- Can support multiple analysis tasks

Potential weaknesses

- Requires suitable data
- Can overfit
- Can learn shortcuts
- Can be difficult to interpret
- May fail with distribution shift

A sophisticated model cannot compensate for poor labels, flawed evaluation, or an unrealistic problem formulation.

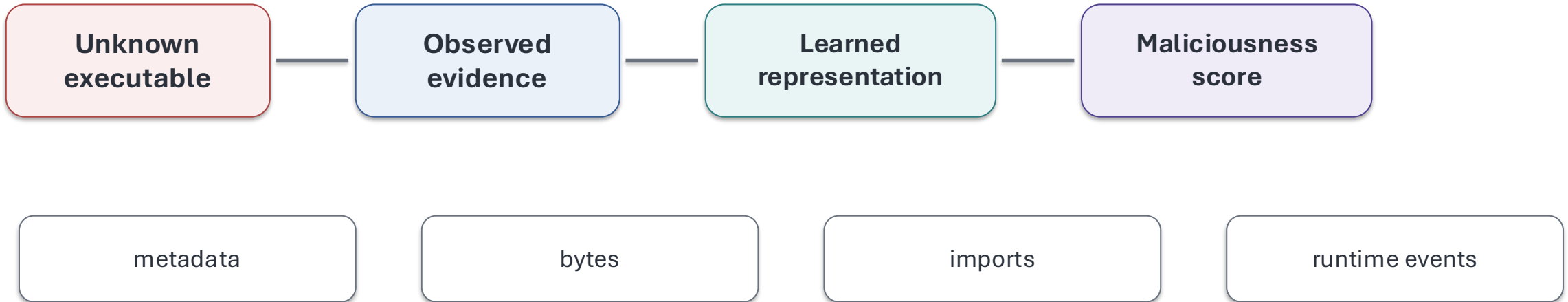
The malware deep-learning problem — abstractly



$$\text{Decision} = f_{\theta}(\text{Representation}(\text{malware}))$$

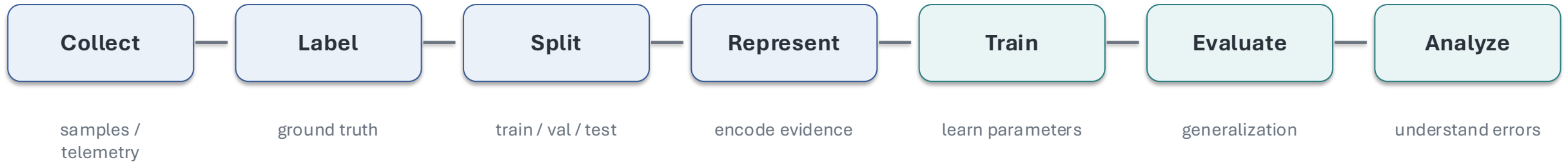
The research problem is not only “which model?” but also “what evidence should the model learn from?”

Worked conceptual example: classifying an unknown executable



- The system receives an executable and observes available evidence.
- That evidence is encoded into a representation suitable for learning.
- The learned function maps the representation to a score or prediction.
- The output can support triage; it does not replace investigation or context.

The experimental pipeline matters as much as the model



If the evaluation protocol is flawed, a high score can be meaningless.

- Related malware samples can make random splitting overly optimistic.
- Depending on the deployment scenario, temporal, family-aware, or deduplicated evaluation may be necessary.

Five questions to ask in every malware-DL study

1

What is the input?

Exactly what evidence is given to the model?

2

What is learned?

Which representation is obtained from that evidence?

3

What is the target?

What decision or quantity is being predicted?

4

Does it generalize?

Would it work on realistic unseen malware?

5

Can we trust the result?

Are labels, splits, metrics, and analysis valid?